

Despliegue automatizado

El despliegue **oficial** del proyecto es en **Kubernetes** y se ejecuta con un solo comando: **make**. Docker Compose existe como alternativa **para desarrollo y pruebas locales** (**make dev-up**).

Kubernetes y Compose nunca deben correr a la vez. Usan los mismos puertos del host (8080, 3000...) y comparten el límite de conexiones de Supabase (15 en el Session Pooler). Por eso cada uno detiene al otro al arrancar: **make** detiene Compose y **make dev-up** elimina el despliegue de Kubernetes.

```
make help           # lista todos los comandos
```

1. Requisitos

Herramienta	Para qué	Nota
Docker	Construir las imágenes	Docker Desktop o Docker Engine
kubectl y un cluster local	Despliegue oficial	Probado con el Kubernetes de Docker Desktop (Settings → Kubernetes → Enable)
make	Ejecutar el Makefile	Funciona con GNU Make 3.81 (la versión de macOS)
openssl	Generar la contraseña del keystore TLS	
Credenciales de Supabase	Ambos caminos	URL, usuario y contraseña del Session Pooler (puerto 5432)

2. Inicio rápido (despliegue oficial)

```
make                # equivale a "make k8s-start"
```

Es un solo comando. En orden, **make**:

1. detiene Docker Compose si está corriendo (para liberar los puertos);
2. crea **dns_api/.env** y **health_checker/.env** desde sus **.env.example**, y agrega las variables nuevas que falten;
3. completa las credenciales de Supabase (ver abajo) y genera el **DATABASE_URL** del Health Checker;
4. construye las 4 imágenes;
5. crea los Secrets y aplica los manifiestos de Kubernetes (los 4 módulos, con los 2 Health Checkers encendidos) y espera a que los pods estén listos;
6. prueba API (HTTP y HTTPS), UI e interceptor;
7. **abre la UI en el navegador:** <http://localhost:30080>.

Credenciales de Supabase. Se toman, en este orden, de:

- **variables de entorno**, útil para automatizar sin terminal interactiva:

```
SUPABASE_DB_URL='jdbc:postgresql://<host>:5432/postgres?
sslmode=require' \
SUPABASE_DB_USER='postgres.<project_ref>' \
SUPABASE_DB_PASSWORD='<password>' \
make
```

- **un prompt en la terminal**: si no están definidas, **make** las pide (la contraseña no se muestra);
- **el archivo `dns_api/.env`**, si ya las escribiste ahí.

Quedan guardadas en `dns_api/.env`, que **no se versiona**: la siguiente vez **make** ya no las pide. Si no hay variables ni terminal, **make** se detiene y explica qué falta.

3. Kubernetes (oficial)

Comando	Qué hace
<code>make / make k8s-start</code>	Todo el flujo anterior: configura, construye, despliega, prueba y abre la UI.
<code>make k8s-up</code>	Lo mismo sin detener Compose, sin probar y sin abrir el navegador.
<code>make k8s-build</code>	Construye <code>dns-api</code> , <code>dns-ui</code> , <code>dns-interceptor</code> y <code>health-checker</code> con el tag <code>:local</code> . La UI recibe <code>NEXT_PUBLIC_DNS_API_URL</code> como <code>--build-arg</code> (variable <code>UI_API_URL</code>).
<code>make k8s-secrets</code>	Crea o actualiza <code>dns-api-secret</code> y <code>health-checker-secret</code> desde los <code>.env</code> . Conserva la contraseña del keystore TLS entre ejecuciones.
<code>make k8s-deploy</code>	Aplica namespace, ConfigMaps, Deployments y Services, reinicia los pods para que usen las imágenes recién construidas y espera a que estén listos.
<code>make k8s-status</code>	Pods y Services del namespace <code>dns</code> .
<code>make k8s-test</code>	Espera al API y prueba API por HTTP y HTTPS, UI e interceptor (resuelve el MX de gmail.com).
<code>make k8s-open</code>	Abre la UI en el navegador (<code>UI_URL</code> , por defecto http://localhost:30080).
<code>make k8s-logs</code> <code>S=dns-api</code>	Últimas 100 líneas de un servicio.
<code>make k8s-checkers-on / make k8s-checkers-off</code>	Enciende o apaga los dos Health Checkers sin redespargar el resto.
<code>make k8s-down</code>	Borra el namespace completo (pods, servicios y secrets).

3.1 Qué se despliega

Módulo	Deployment	Acceso desde el host
DNS API	<code>dns-api</code> (Service <code>LoadBalancer</code>)	<code>http://localhost:8080</code> y <code>https://localhost:8443</code>
DNS UI	<code>dns-ui</code> (<code>NodePort</code>)	<code>http://localhost:30080</code>
DNS Interceptor	<code>dns-interceptor</code> (<code>NodePort</code> , UDP)	<code>nslookup -port=30053 <dominio> 127.0.0.1</code>
Health Checker	<code>health-checker-cr</code> y <code>health-checker-us</code>	(sin puerto; escriben en Supabase)

Dentro del cluster el interceptor llega al API por `https://dns-api:8443` (nombre del Service).

3.2 Configuración: ConfigMap y Secret

Ningún valor de configuración está en el código; todo entra por variables de entorno.

Recurso	Contenido
<code>dns-api-config</code> (ConfigMap)	Puertos, <code>DNS_UI_ORIGIN</code> , DNS remoto y timeout, <code>DB_POOL_SIZE</code> , opciones de HTTPS.
<code>dns-api-secret</code> (Secret)	<code>SUPABASE_DB_URL</code> , <code>SUPABASE_DB_USER</code> , <code>SUPABASE_DB_PASSWORD</code> , <code>SSL_KEYSTORE_PASSWORD</code> .
<code>dns-interceptor-config</code>	<code>DNS_API_URL</code> .
<code>health-checker-config</code>	<code>CHECK_INTERVAL_SECONDS</code> . La ubicación de cada checker (id, país, ciudad, latitud, longitud) va en su Deployment.
<code>health-checker-secret</code>	<code>DATABASE_URL</code> (URI de <code>libpq</code>).

Los Secrets reales **nunca se versionan**: `make k8s-secrets` los crea desde tus `.env`. Los archivos `k8s/*/secret.example.yaml` son solo plantillas de referencia y el `Makefile` no los aplica.

3.3 Health Checkers

Se despliegan **encendidos**: son parte del sistema y cada uno prueba los targets desde su ubicación simulada, `CR-01` (Cartago) y `US-01` (Nueva York), para el tipo `round-trip`.

Escriben en la base compartida: insertan filas en `health_results` y cambian el `healthy` de `records.ips`. Con los targets de ejemplo (`10.0.x.x`, `1.2.3.4`) casi todo pasará a `unhealthy`, y el interceptor no responde con IPs caídas. Para desplegar sin ellos: `make CHECKERS=off`. Para apagarlos o encenderlos después: `make k8s-checkers-off` y `make k8s-checkers-on`.

3.4 Otros clusters

- **kind:** `make KIND_CLUSTER=<nombre>` carga las imágenes en el cluster tras construirlas.
- **minikube:** ejecuta antes `eval $(minikube docker-env)` para construir dentro de su Docker.
- El **Service** del API es **LoadBalancer**, que solo se publica en **localhost** en Docker Desktop. En otros clusters usa `kubectl -n dns port-forward svc/dns-api 8080:8080` o cambia el tipo a **NodePort**.

4. Docker Compose (desarrollo y pruebas locales)

Alternativa para desarrollar sin Kubernetes. **Al levantarse, elimina el despliegue de Kubernetes** (namespace **dns**).

Comando	Qué hace
<code>make env</code>	Crea los <code>.env</code> que falten desde los <code>.env.example</code> y agrega las variables nuevas.
<code>make dev-up</code>	Configura (<code>.env</code> , credenciales, <code>DATABASE_URL</code>) y ejecuta <code>docker compose up --build -d</code> . Con <code>SERVICES="dns-api dns-ui"</code> levanta solo esos servicios.
<code>make dev-test</code>	Espera al API (hasta 2 minutos) y prueba API, UI e interceptor.
<code>make dev-ps / make dev-logs [S=dns-api]</code>	Estado y logs de los contenedores.
<code>make dev-down</code>	Detiene y elimina los contenedores.

Servicios y puertos (host):

Servicio	URL	Nota
DNS API	<code>http://localhost:8080</code>	Solo HTTP (el HTTPS se activa con <code>SSL_ENABLED=true</code> , ver sección 5)
DNS UI	<code>http://localhost:3000</code>	<code>DNS_UI_ORIGIN</code> del API coincide con este origen
DNS Interceptor	UDP <code>localhost:15353</code>	<code>nslookup -port=15353 example.org 127.0.0.1</code>
Health Checker	(sin puerto)	Escribe resultados en Supabase cada <code>CHECK_INTERVAL_SECONDS</code>

El Health Checker de Compose **escribe en Supabase y actualiza el estado `healthy` de los registros**. Con IPs de ejemplo (`10.0.x.x`, `1.2.3.4`) casi todo pasará a `unhealthy`. Para levantar todo menos el checker: `make dev-up SERVICES="dns-api dns-ui dns-interceptor"`.

Los contenedores de Compose tienen `restart: unless-stopped`: si reinicias Docker Desktop **vuelven a arrancar solos** aunque los hayas detenido con otro comando. Si vas a usar Kubernetes, ejecuta `make dev-down` o simplemente `make`.

5. HTTPS del DNS API

El enunciado pide que el interceptor consuma el API por HTTPS. En Kubernetes el API arranca con `SSL_ENABLED=true`: sirve HTTPS en 8443 (para el interceptor) y mantiene HTTP en 8080 (para la UI, porque un navegador no acepta un certificado autofirmado sin intervención). El certificado autofirmado lo genera `docker-entrypoint.sh` al arrancar el contenedor, con la contraseña del Secret. Para probarlo: `curl -k https://localhost:8443/api/records`.

6. Solución de problemas

Síntoma	Causa	Solución
Faltan credenciales de Supabase	No hay variables de entorno ni terminal interactiva para pedir las	Definir <code>SUPABASE_DB_URL</code> , <code>SUPABASE_DB_USER</code> y <code>SUPABASE_DB_PASSWORD</code> al invocar <code>make</code> , o escribirlas en <code>dns_api/.env</code>
No hay un cluster accesible con <code>kubectl</code>	Kubernetes no está activado	Docker Desktop → Settings → Kubernetes → Enable
Pod <code>dns-api</code> en <code>CrashLoopBackOff</code> con <code>EMAXCONNSESSION</code>	El Session Pooler de Supabase admite 15 conexiones en total y está lleno	Cerrar otros clientes: Compose (<code>make dev-down</code>), otro API, checkers, <code>psql</code> . El pool del API es <code>DB_POOL_SIZE</code> (5 en k8s)
Pod <code>dns-api</code> en <code>CrashLoopBackOff</code> con <code>UnknownHostException</code> justo tras reiniciar Docker Desktop	El cluster aún no resolvía nombres	Se recupera solo en unos segundos
<code>dns-api EXTERNAL-IP <pending></code>	El puerto 8080 del host está ocupado (p. ej. por Compose)	<code>make</code> (detiene Compose) o <code>make dev-down</code>
<code>ErrImagePull / ImagePullBackOff</code>	La imagen <code>:local</code> no existe en el cluster	<code>make k8s-build</code> ; con kind, usar <code>KIND_CLUSTER</code>
El API no arranca: <code>Could not resolve placeholder 'DNS_UI_ORIGIN'</code>	Un <code>.env</code> antiguo sin esa variable	<code>make env</code> la agrega desde <code>.env.example</code>
<code>nslookup</code> da timeout con <code>single.example.com</code>	Un Health Checker marcó su IP como <code>unhealthy</code> y el interceptor no responde con IPs caídas	Revisar/restaurar <code>healthy</code> con <code>PUT /api/records/{name}</code>
Fallos ocasionales <code>502</code> en <code>/api/dns_resolver</code>	Docker Desktop pierde ~10 % de paquetes UDP; el API reintenta una vez (<code>DNS_REMOTE_RETRIES</code>)	Aumentar <code>DNS_REMOTE_RETRIES</code>

7. Uso de IA y problemas encontrados

Esta sección documenta el uso de IA generativa en la parte de automatización, como pide el enunciado. Revisar y completar con los prompts propios de cada integrante.

Herramienta: Claude (Sonnet 5 y 5.5) mediante Claude Code, trabajando sobre el repositorio.

Prompts principales (resumidos):

- 1. "Necesito que cada módulo tenga su Dockerfile y en la raíz un docker compose que levante los contenedores de cada módulo. ¿Un Dockerfile es un contenedor? ¿O un docker compose es un contenedor?"
- 2. "Tengo un proyecto con 4 módulos (interceptor en Rust, API en Java, health checker en C, UI en React), cada uno con su Dockerfile y un docker-compose en la raíz. Quiero desplegarlo en Kubernetes local con manifiestos YAML planos, sin Helm. Primero dime qué cluster local recomiendas y cómo cargar las imágenes."
- 3. "Genera el Deployment, el Service, el ConfigMap y el Secret del dns-api con las variables del compose." (y las peticiones equivalentes para dns-interceptor, health-checker y dns-ui, con NodePort fijo, probes válidas para UDP y un Deployment por ubicación con lat, lon, país y ciudad distintos).
- 4. "¿Cómo automatizo Kubernetes?" → "Escríbelo con el Makefile y el docu/k8s.md."
- 5. "Aún no está automatizado" (con la salida de `make`, que solo mostraba la ayuda) → el `Makefile` pasó a ejecutar todo el flujo con un solo comando.
- 6. "Ocupo un comando que levante todo desde k8s, automatizado y abra la URL de la UI directamente." y "no deben estar los dos a la vez: la automatización oficial es la de Kubernetes y la final; la de Compose existe como manera de desarrollo" → `make` despliega Kubernetes y abre la UI; Compose pasó a `make dev-*` y cada stack detiene al otro.

Problemas encontrados y cómo se resolvieron:

Problema	Resolución
Un <code>.gitignore</code> de la raíz ignoraba <code>*/dns_api_mock</code> y ocultaba archivos nuevos	Se quitó esa regla
El build de Maven fallaba porque un test no compilaba (<code>RecordResponse</code> cambió de constructor)	Se corrigió el test
El API no arrancaba: Hibernate rechazaba la columna <code>int[]</code> de <code>targets</code> (<code>ddl-auto: validate</code>)	Se cambió el mapeo a <code>@JdbcTypeCode(SqlTypes.ARRAY)</code>
<code>PUT /api/records</code> daba 500: el código ponía <code>counter = null</code> y la columna es <code>NOT NULL</code>	El contador vale 0 para todos los tipos
El puerto 8080 estaba ocupado por un contenedor viejo de Compose y el <code>LoadBalancer</code> quedaba en <code><pending></code>	Se detuvo el contenedor; ahora <code>make</code> detiene Compose antes de desplegar
El pod nuevo del API entraba en <code>CrashLoopBackOff</code> durante un rolling update (<code>EMAXCONNSESSION</code>): el pooler de Supabase admite 15 conexiones	Estrategia <code>Recreate</code> y <code>DB_POOL_SIZE</code> configurable

Problema	Resolución
Los logs del Health Checker no aparecían en <code>kubectl logs</code> (el <code>printf</code> de C queda en buffer al escribir a un pipe)	El contenedor se ejecuta con <code>stdbuf -oL</code>
Probes: <code>tcpSocket</code> no sirve para UDP	El interceptor usa una probe <code>exec</code> sobre <code>/proc/net/udp</code>
Los Health Checkers marcaron <code>unhealthy</code> casi todos los registros de ejemplo y el DNS dejó de resolverlos	Se restauraron los flags. Los checkers son parte del sistema, así que se despliegan encendidos (<code>make CHECKERS=off</code> los omite)
El Health Checker no se creaba con <code>make</code> : se había dejado apagado por defecto	<code>make</code> los despliega encendidos
Tras cada despliegue se veían contenedores duplicados durante ~35 s (interceptor y health-checker): el proceso corre como PID 1 sin manejar <code>SIGTERM</code> y Kubernetes espera los 30 s de gracia antes de matarlo	Solo es visual; se puede reducir con <code>terminationGracePeriodSeconds</code>
Docker Desktop pierde ~10 % de paquetes UDP hacia el DNS remoto (<code>502</code> intermitentes)	Reintentos en el cliente UDP del API
Un <code>.env</code> local antiguo no tenía <code>DNS_UI_ORIGIN</code> y el API de Compose no arrancaba	<code>make env</code> agrega las variables nuevas de los <code>.env.example</code>
Kubernetes y Compose corriendo a la vez se estorbaban (puertos y conexiones a Supabase), y al reiniciar Docker Desktop los contenedores de Compose volvían solos	Kubernetes es el camino oficial; cada stack detiene al otro
<code>make</code> a secas solo mostraba la ayuda, y había que crear y editar los <code>.env</code> a mano	<code>make</code> ejecuta todo el flujo; las credenciales se leen de variables de entorno o se piden por consola
El <code>WORKDIR</code> del Dockerfile del Health Checker cambió a <code>/health_checker</code>	Los manifiestos usan rutas relativas y no se vieron afectados